

Maritime CargoService: Sprint 0

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

June 20, 2026

1. Introduction

Una compagnia di trasporto marittimo di container (d'ora in poi la committente) intende automatizzare le operazioni di carico dei container nella stiva della nave (d'ora in poi hold). A tal fine prevede di impiegare un robot a guida differenziale (Differential Drive Robot, d'ora in poi cargorobot).

L'obiettivo dello Sprint 0 è formalizzare i requisiti forniti dalla committente in modo preciso e non ambiguo, costruire un primo modello logico dei macro-componenti del sistema, evidenziare il core business, motivare la scelta del linguaggio di modellazione e definire un primo insieme di piani di test funzionali.

Ogni scelta è strettamente ancorata ai requisiti: il modello qui presentato serve a catturare il comportamento richiesto, senza anticipare decisioni progettuali o scelte implementative che verranno affrontate negli sprint successivi.

La traccia del progetto può essere scaricata dal seguente link^o

2. Requirements

2.1. Requisiti funzionali

- Il servizio da realizzare è cargoservice: il customer gli invia una richiesta di carico tramite il pushbutton dell'IOPort.
- L'IOPort è un dispositivo di ingresso/uscita dotato di:
 1. Pushbutton, premuto dal cliente per inviare una richiesta di caricamento di un container sul carico.
 2. Display, utilizzato per mostrare la risposta alla richiesta e per mostrare lo stato attuale del blocco.
- Se l'IOPort è occupata da un container o il sistema è Out of service, cargoservice risponde retrylater.
- Se tutti gli slot1–slot4 (ciascuno capace di contenere un container) sono occupati, cargoservice rifiuta la richiesta.
- Se l'IOPort è libera, il sistema è in servizio e almeno uno tra slot1–slot4 è libero, cargoservice entra nello stato engaged, riserva uno slot libero senza criteri di ottimizzazione e restituisce al customer il nome dello slot riservato.
- Fintanto che il sistema è engaged, un LED lampeggia.
- Dopo l'accettazione, il customer ha un tempo prefissato (es. 30 s) per depositare il container nell'area del sonar.
- Se il customer non deposita entro il timeout, il sistema torna disengaged.

- Quando la presenza del container è confermata dal sonar, cargoservice comanda al cargorobot di spostare il container dall'IOPort allo slot5 (adibito al deposito temporaneo di un container prima della collocazione definitiva).
- Il marker device etichetta il container in slot5 e segnala il completamento.
- Alla ricezione del segnale di completamento da parte del marker device, cargoservice comanda al cargorobot di spostare il container da slot5 allo slot riservato.
- Il display dell'IOPort mostra in ogni momento:
 1. Lo stato corrente della hold.
 2. Il messaggio "Service working" durante il normale funzionamento.
 3. Il messaggio "Out of service" se il sonar misura $D > D_{\text{FREE}}$ per ≥ 3 s.
- Il sonar rileva la presenza di un container quando misura $D < D_{\text{FREE}}/2$ per ≥ 3 s.

2.2. Requisiti non funzionali

- La hold è un'area rettangolare piatta contenente:
 1. 4 slots per contenere i container (slot1—slot4).
 2. 1 slot speciale (slot5).
- Il cargorobot è l'entità software/robotica che dovrà governare un DDR; i requisiti non stabiliscono ancora quale parte sia già disponibile e quale debba essere realizzata.
- Il tempo massimo di attesa per il deposito e la soglia di rilevazione sonar sono parametri prefissati (es. 30 s e 3 s rispettivamente).
- Il Sensore è un dispositivo di rilevamento (un sonar) associato all'IOPort. Il suo compito è quello di rilevare la presenza di container da caricare.

2.3. Domande aperte alla committente

Domanda al committente.

Posizione dell'IOPort nella mappa. I requisiti indicano che il cargorobot sposta il container dall'IOPort a slot5, lasciando intendere che l'IOPort sia una cella praticabile. Come si colloca nella griglia?

Domanda al committente.

Richieste concorrenti. Il sistema deve bufferizzare più richieste di carico contemporanee, oppure una nuova richiesta ricevuta mentre il sistema è engaged viene semplicemente rifiutata / respinta con retrylater senza accodamento? Dai requisiti si interpreta che le richieste non siano bufferizzate: se il sistema è occupato, la risposta è immediata (retrylater o refused) senza code di attesa.

Domanda al committente.

Liberazione degli slot e aggiornamento della disponibilità. I requisiti descrivono il processo di carico dei container negli slot1-4, ma non specificano come venga gestita la liberazione di uno slot già occupato. Possiamo assumere che lo svuotamento degli slot sia effettuato da sistemi o operatori esterni al nostro sistema? In tal caso, con quale meccanismo viene notificato a cargoservice che uno specifico slot è stato liberato e può tornare disponibile per future prenotazioni?

Domanda al committente.

Ripristino dello stato Service working. I requisiti specificano che il sistema entra nello stato Out of service quando il sonar rileva una distanza $D > D_{FREE}$ per almeno 3 secondi. Non è invece specificata la condizione per il ritorno allo stato Service working. Il sistema deve tornare operativo non appena il sonar rileva $D \leq D_{FREE}$ oppure è richiesto un ulteriore intervallo di stabilità prima del ripristino del servizio?

3. Requirement analysis

3.1. Core business

Il core business del sistema è la gestione del ciclo di carico di un container. La responsabilità della sequenza applicativa resta in cargoservice: il cargorobot è coinvolto per eseguire spostamenti richiesti dal servizio, non per decidere se una richiesta debba essere accettata, rifiutata o sospesa. La sequenza principale ricavata dai requisiti è:

1. Il customer preme il pushbutton dell'IOPort.
2. cargoservice verifica le precondizioni (stato sistema, IOPort, disponibilità slot).
3. Se soddisfatte: stato engaged, prenotazione slot, notifica al customer.
4. Il customer deposita il container nell'area del sonar entro il timeout.
5. cargoservice comanda al cargorobot di spostare il container da IOPort a slot5.
6. Il marker device etichetta il container e segnala il completamento.
7. cargoservice comanda al cargorobot di spostare il container da slot5 allo slot riservato; il sistema torna disengaged.

3.2. Macro-componenti e natura software

COMPONENTE	RUOLO	STATO
cargoservice	Orchestratore principale. Natura reattiva (richieste in arrivo) e proattiva (comandi al cargorobot, aggiornamenti display). Non può restare privo di controllo: da requi-	Da sviluppare

	sito deve rispondere e agire autonomamente.	
cargorobot	Entità che governa il DDR per la movimentazione fisica richiesta dal cargoservice. La scelta POJO vs microservizio è rimandata all'analisi: se fosse un POJO, il cargoservice gli cederebbe il controllo (trasferimento di controllo sincrono) e non potrebbe reagire ad altri eventi durante la movimentazione. Se invece fosse un microservizio, la comunicazione sarebbe asincrona e message-driven, con un disaccoppiamento molto maggiore. Forti motivazioni spingono verso la seconda opzione, ma la decisione è oggetto degli sprint successivi.	Da chiarire / da sviluppare
IOPort	Interfaccia con il customer (pushbutton + display). Se realizzata come microservizio separato (rappresentata per esempio tramite una Single Page Application (SPA), il pushbutton come un button HTML e il display come textarea HTML.), le responsabilità sono completamente disaccoppiate da cargoservice e i due possono essere sviluppati in parallelo; l'unica interfaccia condivisa è il messaggio load_request definito in questo sprint.	Dispositivo fisico fornito sw da sviluppare
sonar	Sensore di distanza.	Dispositivo fisico fornito driver sw da sviluppare
marker device	Etichettatura in slot5.	Dispositivo fisico simulato fornito sw da sviluppare

LED	Indicatore stato engaged. Può essere fisico o virtuale (rappresentato ad esempio come un indicatore visivo lampeggiante, eventualmente nella stessa SPA dell'IOPort). La scelta dipende dall'infrastruttura disponibile.	Fisico o virtuale sw da sviluppare
hold	Struttura dati per lo stato della stiva (occupazione slot). Passivo.	Da sviluppare

3.3. Motivazione dell'uso del linguaggio QAK

QAK è il linguaggio messo a disposizione dalla nostra software house (unibo.issLab) per modellare sistemi software distribuiti, particolarmente espressivo nel formalizzare il concetto di attore autonomo e di messaggio, riducendo di molto l'“Abstraction Gap” tra requisiti e modello. La natura reattiva e proattiva di cargoservice, che deve rispondere a stimoli esterni e avviare autonomamente sequenze di azioni, è infatti catturata in modo naturale da un attore QAK, cosa che un POJO (Plain Old Java Object), componente passivo attivato da chiamate sincrone, non catturerebbe altrettanto bene. Dal modello QAK viene generato automaticamente codice Kotlin eseguibile, il che consente di disporre di un primo prototipo osservabile già nello Sprint 0, prima ancora di scrivere una riga di logica applicativa.

3.4. Formalizzazione dei messaggi QAK

La seguente formalizzazione non introduce ancora scelte di progetto, limitandosi ad elencare i messaggi necessari per esprimere i requisiti. Request/Reply viene usato quando il requisito prevede una risposta osservabile; Dispatch quando il requisito parla di una segnalazione o di un aggiornamento senza risposta diretta.

3.4.1. customer / IOPort -> cargoservice

```
Request load_request      : loadRequest(none)
Reply   load_accepted     : loadAccepted(slotID)   for load_request
Reply   load_retrylater    : loadRetryLater(none)   for load_request
Reply   load_refused       : loadRefused(none)      for load_request
```

3.4.2. sonar -> cargoservice

```
Dispatch sonar_data       : sonarData(distance)
Dispatch container_detected : containerDetected(none)
Dispatch sonar_failure     : sonarFailure(none)
```

Nota: La scelta Dispatch (e non Request) per i messaggi del sonar è una ipotesi di analisi: il sonar segnala eventi al cargoservice senza aspettarsi una risposta diretta. Se l'analisi rivelasse la necessità di una conferma (es. handshake di rilevazione), il tipo del messaggio andrà rivisto nello sprint 1.

3.4.3. cargoservice -> cargorobot

```
Request  move_to_slot5 : moveToSlot5(none)
Reply    move_done      : moveDone(none)      for move_to_slot5

Request  move_to_slot   : moveToSlot(slotID)
Reply    move_done      : moveDone(none)      for move_to_slot
```

3.4.4. marker device -> cargoservice

```
Dispatch marking_done : markingDone(containerID)
```

3.4.5. cargoservice -> marker device

```
Dispatch start_marking : startMarking(containerID)
```

3.4.6. cargoservice -> display / LED

```
Dispatch display_hold_state : holdState(state)
Dispatch display_status     : displayStatus(message)
Dispatch led_on              : ledOn(none)
Dispatch led_off             : ledOff(none)
```

3.5. Contesti logici

CONTESTO	COMPONENTI E RESPONSABILITÀ
ctxCargoService	Contiene l'attore cargoservice. Nucleo del comportamento richiesto e punto di orchestrazione del ciclo di carico.
ctxIO	Raggruppa le entità legate all'IOPort e ai dispositivi citati dai requisiti: ioport, sonar, led, markerdevice.
ctxRobot	Raggruppa le entità legate al cargorobot e alla movimentazione richiesta.

3.6. Schema della hold

Legenda: H = HOME S = sonar/sensor area 1-4 = slot1-4
5 = slot5 I = IOPort X = ostacolo . = libero

```
col0 col1 col2 col3 col4 col5 col6
riga0 [ H ][ S ][ . ][ . ][ . ][ . ][ . ]
```

```

riga1 [ . ][ 1 ][ X ][ X ][ 2 ][ . ][ . ]
riga2 [ . ][ . ][ . ][ . ][ X ][ 5 ][ . ]
riga3 [ . ][ 3 ][ X ][ X ][ 4 ][ . ][ . ]
riga4 [ . ][ . ][ . ][ . ][ . ][ . ][ . ]
riga5 [ I ][ X ][ X ][ X ][ X ][ X ][ X ]

```

Nota: La posizione esatta dell'IOPort e del sonar richiede conferma dalla committente. Lo schema è una prima interpretazione fedele alla figura allegata ai requisiti.

4. Test plan

I test funzionali verificano il comportamento osservabile del sistema rispetto ai requisiti, indipendentemente dall'implementazione.

SCENARIO	PRECONDIZIONI	RISULTATO ATTESO
Accettazione	disengaged, non OoS, IOPort libera, slot disponibile	load_accepted con slot riservato; stato engaged; LED lampeggiante
Rifiuto (hold piena)	disengaged, non OoS, IOPort libera, slot1-4 tutti occupati	load_refused; stato disengaged; LED spento
Retrylater (OoS)	Sistema Out of service	load_retrylater; stato immutato
Retrylater (IOPort occupata)	disengaged, non OoS, IOPort occupata da un container	load_retrylater; stato disengaged
Timeout deposito	Sistema engaged; customer non deposita entro il tempo prefissato	Sistema disengaged; LED spento
Ciclo completo	disengaged, non OoS, IOPort libera, slot disponibile	Container nello slot riservato; display "Service working"; disengaged; LED spento
Guasto sonar	Sistema operativo; sonar misura $D > D_{FREE}$ per ≥ 3 s	Sistema Out of service; display "Out of service"
Rilevazione container	Sistema engaged; sensor area vuota; sonar misura $D < D_{FREE}/2$ per ≥ 3 s	cargoservice avvia la movimentazione verso slot5

5. Project

Dall'analisi dei requisiti si propone, come ipotesi iniziale, una struttura a tre sprint con integrazione progressiva dei componenti. La suddivisione serve a organizzare il lavoro e i test, non a fissare decisioni architetturali definitive.

5.1. Sprint 1: Core business

Goal: realizzare il primo nucleo eseguibile di cargoservice con collaboratori simulati.

Funzionalità: ciclo di carico completo con collaboratori simulati, modello dello stato della hold, LED simulato, timeout.

5.2. Sprint 2: IOPort, display e sonar

Goal: rendere esplicita l'interazione con IOPort, display e sonar a livello software.

Funzionalità: ioport come interfaccia web, sonar simulato (rilevazione container e malfunzionamento), aggiornamento display con stato e messaggi.

5.3. Sprint 3: Dispositivi fisici

Goal: integrare, dove disponibili, i dispositivi fisici o i servizi forniti e verificare il comportamento end-to-end.

Funzionalità: cargorobot e servizio DDR disponibile, sonar fisico, marker device fisico, LED fisico.

6. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340